



RTL LABORATORY FILE

SUBMITTED BY:

NAME	ABDUL RAFAY
------	-------------

SUBMITTED TO:

SIR KHALIL REHMAN

(RTL) WEEK # 04 LAB

Table Of Contents

AXI4-LITE (MASTER AND SLAVE)	-----3
-------------------------------------	---------------

RTL LAB WEEK # 04

DESIGN AND SIMULATION OF AXI4-LITE MASTER AND SLAVE

➤ OBJECTIVE:

To implement and simulate AXI4-Lite Master and Slave modules for performing memory read and write transactions using the AXI4-Lite communication protocol.

LAB TASKS

➤ Question 1: Design and simulate a AXI4-Lite Master and Slave using SystemVerilog.

❖ CODE:

• DESIGN CODE:

➤ AXI_LITE_MASTER.SV:

```
// ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 04 (RTL)
// CODE TASK # 02
// AXI LITE MASTER
module axi_lite_master (
    input logic    clk,
    input logic    rst_n,
    input logic    start,
    input logic    wr_rd,
    input logic [7:0] addr,
    input logic [31:0] wdata,
    output logic [31:0] rdata,
    output logic    done,
    output logic    awvalid,
    input logic    awready,
    output logic [7:0] awaddr,
    output logic    wvalid,
    input logic    wready,
    output logic [31:0] wdata_out,
    output logic [3:0] wstrb,
    output logic    bready,
    input logic    bvalid,
    input logic [1:0] bresp,
    output logic    arvalid,
    input logic    arready,
    output logic [7:0] araddr,
    input logic    rvalid,
    output logic    rready,
    input logic [31:0] rdata_in,
    input logic [1:0] rresp
```

```

);
typedef enum logic [1:0] {IDLE, WRITE, READ} state_t;
state_t state, next_state;
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        state <= IDLE;
    else
        state <= next_state;
end
always_comb begin
    next_state = state;
    case (state)
        IDLE: begin
            if (start && wr_rd)
                next_state = WRITE;
            else if (start && !wr_rd)
                next_state = READ;
            end
        WRITE: if (bvalid && bready) next_state = IDLE;
        READ:  if (rvalid && rready) next_state = IDLE;
    endcase
end
always_comb begin
    awvalid = 1'b0;
    awaddr = 8'd0;
    wvalid = 1'b0;
    wdata_out = 32'd0;
    wstrb = 4'hF;
    bready = 1'b0;
    arvalid = 1'b0;
    araddr = 8'd0;
    rready = 1'b0;
    rdata = 32'd0;
    done = 1'b0;
    case (state)
        IDLE: begin
            awaddr = addr;
            araddr = addr;
            wdata_out = wdata;
        end
        WRITE: begin
            awvalid = 1'b1;
            awaddr = addr;
            if (awready) begin
                wvalid = 1'b1;
                wdata_out = wdata;
            end
            if (wready) begin
                bready = 1'b1;
                if (bvalid && bready) begin
                    done = 1'b1;
                end
            end
        end
    endcase
end

```

```

        end
    end
end
READ: begin
    arvalid = 1'b1;
    araddr = addr;
    if (arready) begin
        rready = 1'b1;
        if (rvalid && rready) begin
            rdata = rdata_in;
            done = 1'b1;
        end
    end
end
endcase
end
endmodule

```

➤ AXI_LITE_SLAVE.SV:

```

// ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 04 (RTL)
// CODE TASK # 02
// AXI LITE SLAVE
module axi_lite_slave (
    input logic    clk,
    input logic    rst_n,
    input logic    awvalid,
    output logic    awready,
    input logic [7:0] awaddr,
    input logic    wvalid,
    output logic    wready,
    input logic [31:0] wdata,
    input logic [3:0] wstrb,
    input logic    bready,
    output logic    bvalid,
    output logic [1:0] bresp,
    input logic    arvalid,
    output logic    arready,
    input logic [7:0] araddr,
    output logic    rvalid,
    input logic    rready,
    output logic [31:0] rdata,
    output logic [1:0] rresp
);
    logic [31:0] mem [0:255];
    typedef enum logic [1:0] {IDLE, WR_RESP, RD_RESP} state_t;
    state_t state, next_state;
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)

```

```

        state <= IDLE;
    else
        state <= next_state;
    end
always_comb begin
    next_state = state;
    case (state)
        IDLE: begin
            if (awvalid && wvalid)
                next_state = WR_RESP;
            else if (arvalid)
                next_state = RD_RESP;
            end
        WR_RESP: if (bready) next_state = IDLE;
        RD_RESP: if (rready) next_state = IDLE;
    endcase
end
always_comb begin
    awready = 1'b0;
    wready = 1'b0;
    bvalid = 1'b0;
    bresp = 2'b00;
    arready = 1'b0;
    rvalid = 1'b0;
    rdata = 32'd0;
    rresp = 2'b00;
    case (state)
        IDLE: begin
            awready = 1'b1;
            arready = 1'b1;
        end
        WR_RESP: begin
            if (awvalid && wvalid) begin
                mem[awaddr] = wdata;
                bvalid = 1'b1;
                bresp = 2'b00;
            end
        end
        RD_RESP: begin
            if (arvalid) begin
                rdata = mem[araddr];
                rvalid = 1'b1;
                rresp = 2'b00;
            end
        end
    endcase
end
endmodule

```

➤ TOP_AXI_LITE.SV:

```
// ABDUL RAFAY
```

```

// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 04 (RTL)
// CODE TASK # 02
// TOP AXI LITE
module top_axi_lite (
    input logic    clk,
    input logic    rst_n,
    input logic    start,
    input logic    wr_rd,
    input logic [7:0] addr,
    input logic [31:0] wdata,
    output logic [31:0] rdata,
    output logic    done
);
    logic    awvalid;
    logic    awready;
    logic [7:0] awaddr;
    logic    wvalid;
    logic    wready;
    logic [31:0] wdata_out;
    logic [3:0] wstrb;
    logic    bready;
    logic    bvalid;
    logic [1:0] bresp;
    logic    arvalid;
    logic    arready;
    logic [7:0] araddr;
    logic    rvalid;
    logic    rready;
    logic [31:0] rdata_in;
    logic [1:0] rresp;
    axi_lite_master master (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .wr_rd(wr_rd),
        .addr(addr),
        .wdata(wdata),
        .rdata(rdata),
        .done(done),
        .awvalid(awvalid),
        .awready(awready),
        .awaddr(awaddr),
        .wvalid(wvalid),
        .wready(wready),
        .wdata_out(wdata_out),
        .wstrb(wstrb),
        .bready(bready),
        .bvalid(bvalid),
        .bresp(bresp),

```

```

        .arvalid(arvalid),
        .arready(arready),
        .araddr(araddr),
        .rvalid(rvalid),
        .rready(rready),
        .rdata_in(rdata_in),
        .resp(rresp)
    );
    axi_lite_slave slave (
        .clk(clk),
        .rst_n(rst_n),
        .awvalid(awvalid),
        .awready(awready),
        .awaddr(awaddr),
        .wvalid(wvalid),
        .wready(wready),
        .wdata(wdata_out),
        .wstrb(wstrb),
        .bready(bready),
        .bvalid(bvalid),
        .bresp(bresp),
        .arvalid(arvalid),
        .arready(arready),
        .araddr(araddr),
        .rvalid(rvalid),
        .rready(rready),
        .rdata(rdata_in),
        .resp(rresp)
    );
endmodule

```

• TEST BENCH CODE:

```

// ABDUL RAFAY
// PSEB USTP INSPIRE SESSION
// SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
// WEEK # 04 (RTL)
// CODE TASK # 02
// TEST-BENCH FOR AXI LITE
module tb_top_axi_lite;
    logic    clk;
    logic    rst_n;
    logic    start;
    logic    wr_rd;
    logic [7:0]  addr;
    logic [31:0] wdata;
    logic [31:0] rdata;
    logic    done;
    top_axi_lite dut (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),

```

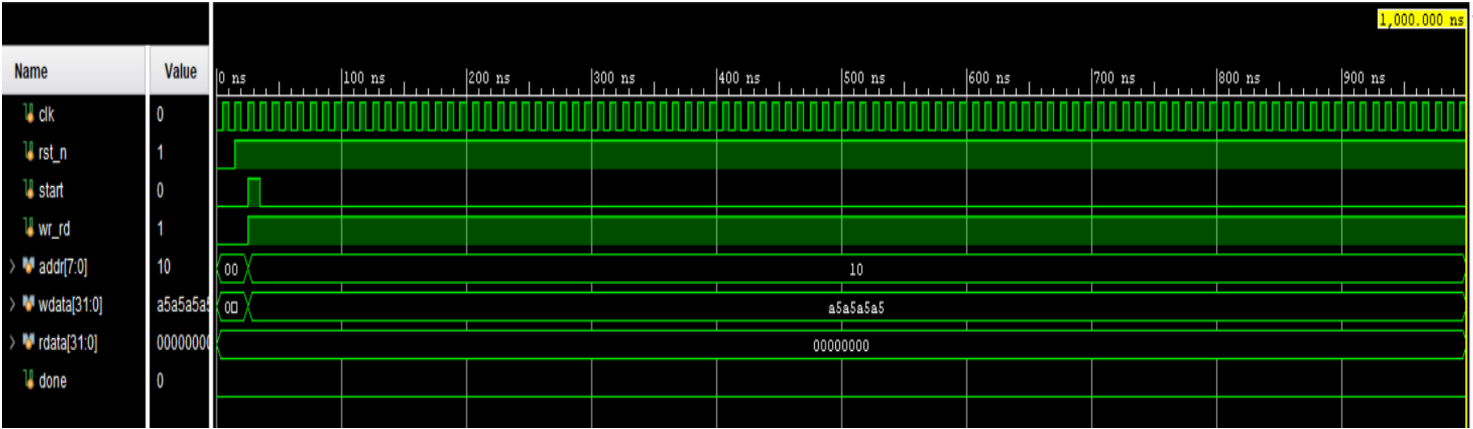


```

.wr_rd(wr_rd),
.addr(addr),
.wdata(wdata),
.rdata(rdata),
.done(done)
);
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
initial begin
    $dumpfile("axi_lite.vcd");
    $dumpvars(0, tb_top_axi_lite);
end
initial begin
    rst_n = 1'b0;
    start = 1'b0;
    wr_rd = 1'b0;
    addr = 8'd0;
    wdata = 32'd0;
    #15 rst_n = 1'b1;
    #10;
    wr_rd = 1'b1;
    addr = 8'h10;
    wdata = 32'hA5A5A5A5;
    start = 1'b1;
    #10 start = 1'b0;
    wait(done);
    #10;
    wr_rd = 1'b1;
    addr = 8'h20;
    wdata = 32'h12345678;
    start = 1'b1;
    #10 start = 1'b0;
    wait(done);
    #10;
    wr_rd = 1'b0;
    addr = 8'h10;
    start = 1'b1;
    #10 start = 1'b0;
    wait(done);
    #10;
    wr_rd = 1'b0;
    addr = 8'h20;
    start = 1'b1;
    #10 start = 1'b0;
    wait(done);
    #20;
    $finish;
end
endmodule

```

❖ **WAVE FORM OUTPUT:**



❖ **SCHEMATIC OUTPUT:**

